제24회 임베디드SW경진대회 · 모빌리티 · 결선

IcePredict

블랙아이스를 미리 보고,
놓치면 노면을 재서 제때 세운다

팀 ARTS · 남윤상 · 김진찬 · 이지성

STM32N6 · ThreadX · Raspberry Pi 5 · CARLA HIL · Moza



문제

블랙아이스는 안 보이고, 지금 차는 마른 노면 기준으로 선다



정답이 블랙아이스인 실사진 — 카메라는 '정상'으로 봤다

3.4 %

실사진 얼음 6,340장 중
어떤 문턱으로도 못 잡는 몫

38 → 105 m

60 km/h 정지에 필요한 거리
마른 노면 → 블랙아이스 (μ 0.16)

지금 기술의 빈틈

ABS · ESC — 미끄러지는 바퀴를 잡는다. 노면 마찰을 다른 장치에 알리지 않는다.

긴급제동 · 간격 유지 — 앞차 거리로 제동 시점을 정한다. 마찰을 모르면 마른 노면 기준이라 빙판에서 늦다.

카메라 — 투명한 얼음은 아스팔트와 구분이 어렵다. 반드시 놓치는 경우가 남는다.

해법

이중 방어 — 1차는 미리 보고, 2차는 재서 제때 세운다

1차

카메라가 미리 본다

노면 4분류 + 반사도 + 기상 맥락 → 위험도.
빙판에 닿기 전에 경고·감속.

2차

노면을 재서 제동 시점을 고친다

카메라가 애매하거나 결빙 취약 구간이면 0.4 초 '노면 확인 탭'으로 마찰을 재고, 20 ms 마다 정지 거리를 다시 계산한다.

RTOS

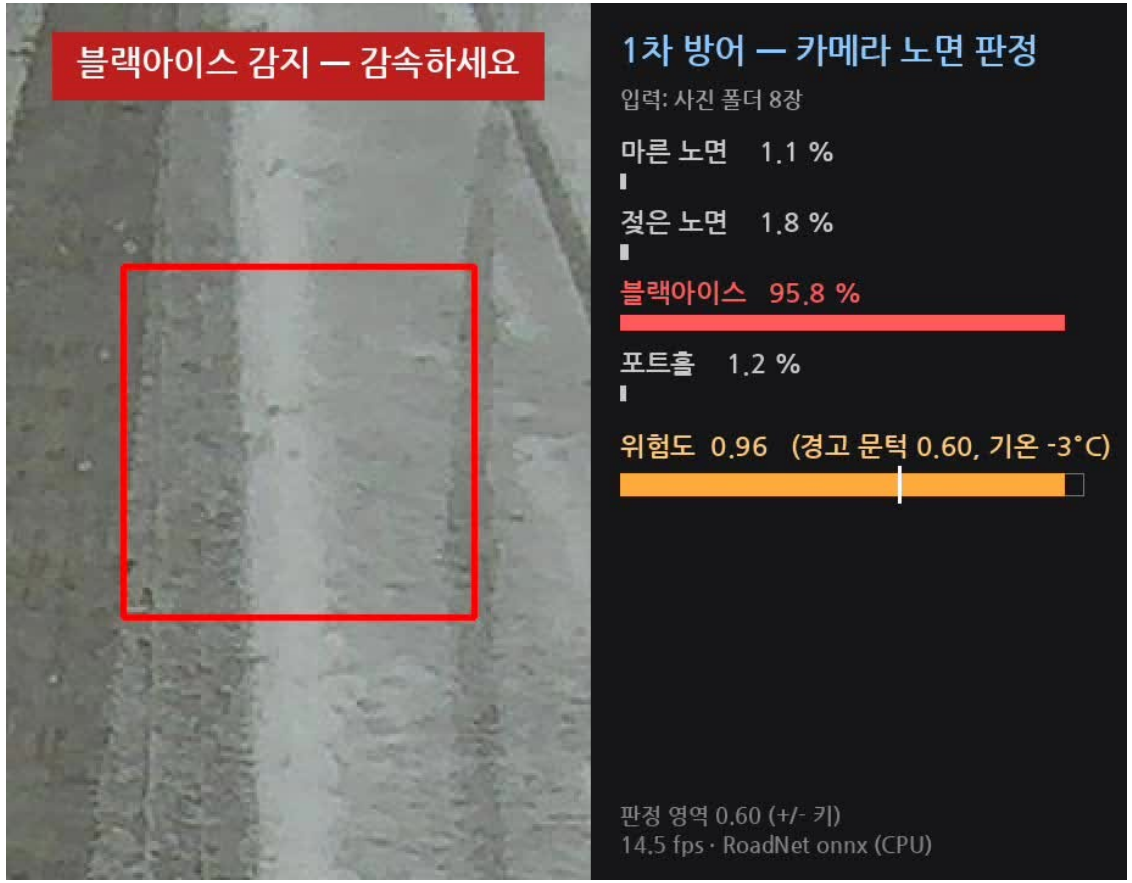
메인 컴퓨터가 죽어도 산다

2차 판단은 리눅스와 분리된 안전 전용 RTOS 에서 돈다. 메인 프로그램이 멈춰도 제동 판단은 계속된다.

조향은 운전자 몫이다. 시스템은 **경고 · 가속 제한 · 제동**만 돕는다 — 자율주행이 아니라 운전자 보조 장치다.

카메라가 빙판을 미리 본다 — 실제 도로 사진 69,358장

▶ 시연 ① RealSense



RealSense 로 빙판 사진을 비추면 실시간으로 판정·경고한다

96.6 %
블랙아이스 실사진 6,340장
경보율

0.3 %
마른 노면 19,018장
오경보율 (젖은 노면 0.8 %)

어떻게

시뮬레이션 화면이 아니라 실제 도로 사진을 STM32N6 보드의 NPU 에 직접 넣어 쟀다.

블랙아이스를 직접 맞히려 하지 않는다. '결빙 위험 확률 + 반사도 + 기상 맥락'으로 위험도를 올리고, 문턱(0.60)을 넘으면 경고한다.

놓치면 — 노면을 재서 '언제부터 서야 하나'를 다시 계산한다

▶ 시연 ② Moza

01 노면 확인 탭

카메라가 애매하거나 영하의 결빙 취약 구간이면 브레이크를 0.4 초 살짝.
빙판은 감속이 거기서 막힌다 — 그 값이 마찰이다.

02 표 조회

CARLA 에서 미리 잰 마찰별 감속 표로 정지 거리를 계산한다. 무거운 계산은 오프라인, 보드는 표만 본다.

03 20 ms 마다 판단

앞 물체 거리와 비교해 경고 → 가속 금지 → 예비 제동 → 전력 제동. 남은 거리에 맞춰 앞차 4 m 뒤에 선다.

정지 차량 앞에서 제동을 시작해야 하는 거리 (m) — 2차 방어가 20 ms 마다 계산하는 값

	30 km/h	40 km/h	50 km/h	60 km/h	70 km/h	80 km/h
μ 0.03 매끈한 얼음	124	216	334	478	648	845
μ 0.07	67	114	176	250	338	439
μ 0.10	47	79	119	169	228	295
μ 0.16 블랙아이스	31	50	75	105	141	182
μ 0.25 다진 눈 정도	23	36	53	73	97	125
μ 0.41 젖은 노면 정도	16	25	35	48	63	80
μ 0.67 마른 노면	14	20	28	38	49	61

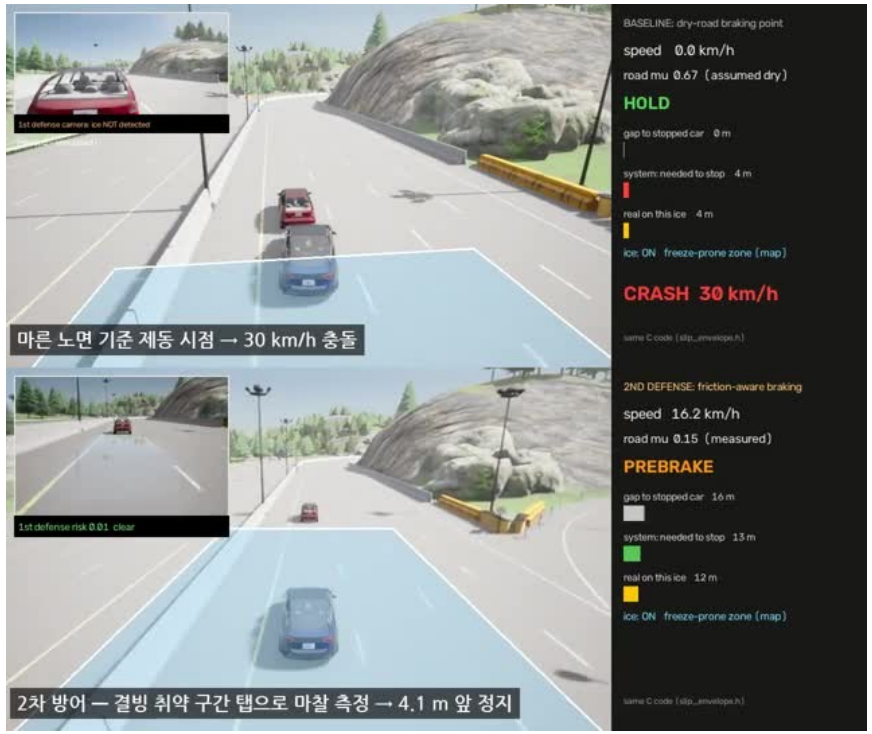
CARLA 0.9.16 실측 감속(제동 1.0) 기준 · 지연 0.3 s · 표 감속의 90 % · 여유 4 m. 비교 대상(마른 노면 기준)은 맨 아래 줄만 쓴다.

제동을 시작해야 하는 거리 (m) — 지금 차는 맨 아랫줄만 쓴다

두 차는 똑같이 미끄러진다.
다른 것은 제동을 시작하는 지점이다.

결과

같은 코드, 마찰만 다르다 — 정지 2 / 9 → 8 / 9



위: 마른 노면 기준 → 30 km/h 충돌 · 아래: 탭으로 마찰 측정 → 4.1 m 앞 정지

2 / 9

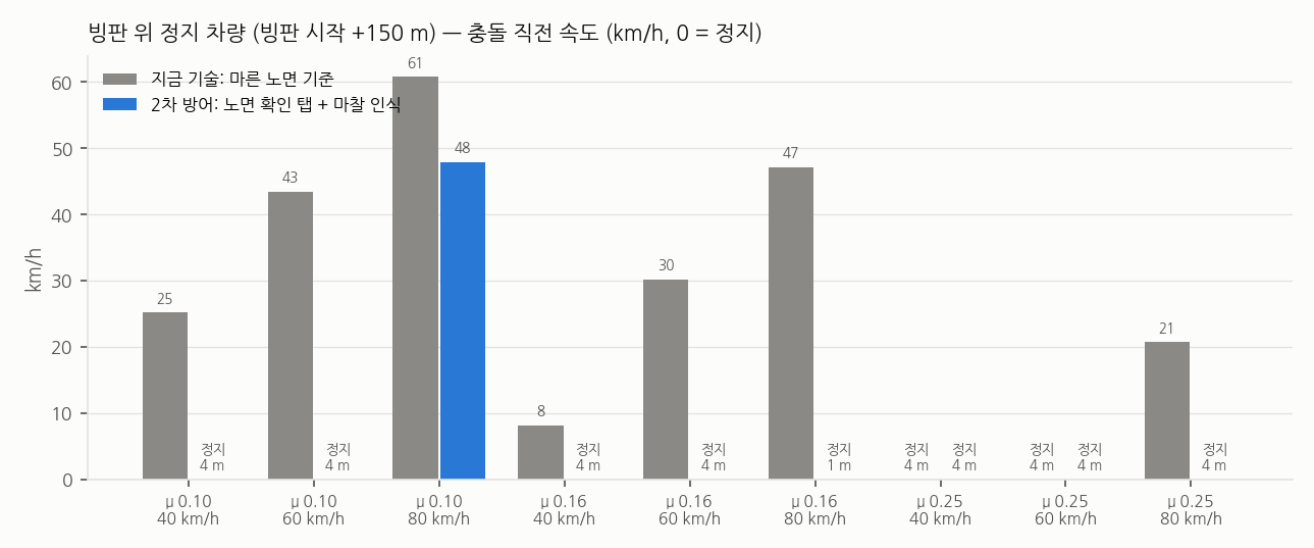
마른 노면 기준 — 정지

나머지 7회 충돌 (8 ~ 61 km/h)

8 / 9

마찰 인식 — 정지

7회는 목표대로 앞차 4 m 앞



마찰 3종 × 속도 3종 · CARLA 0.9.16 · 간격 유지 크루즈 주행 중 빙판 위 정지 차량

안전 전용 RTOS

메인 컴퓨터가 멈춰도 2차 방어는 산다

▶ 시연 ③ 강제 종료



빙판 확인 직후 파이 메인 프로그램을 실제로 강제 종료 → 격리 코어의 2차 방어가 혼자 4.1 m 앞에 세운다

53 → 39 km/h

곡선 빙판에서 파이 정지 시 충돌 속도
2차가 파이 → 2차가 안전 전용 RTOS

0.12 s

메인 심장박동 끊김 감지
→ 축소 모드로 계속 판단

전 코어 부하 · 20 ms × 6,000회	최악 지연	마감 놓침
일반 리눅스 프로그램	206 ms	211
파이 격리 코어 (실시간 · RTOS 대체)	0.06 ms	0

정직하게

설계는 STM32N6 + ThreadX 다. 예전에 보드에서 잰 2차 방어 최악 응답은 22.6 μs.

이번 확장은 보드가 없어 **같은 C 코드**를 라즈베리파이 격리 코어에서 돌렸다. 리눅스 커널 자체가 멈추면 같이 멈춘다 — 그래서 실차는 별도 MCU 다.

정리

미리 보고, 놓치면 재서, 죽어도 판단한다

미리 본다
실사진 블랙아이스 경보율 96.6 %, 마른 노면 오경보 0.3 %

재서 제때 선다
노면 확인 탭 + 마찰 인식 제동 시점 — CARLA 정지 2/9 → 8/9

죽어도 판단한다
메인 컴퓨터가 멈춰도 2차 방어는 계속 — 충돌 53 → 39 km/h

남은 것

· 마찰 측정 탭을 운전자가 느낀다 — 앞축 구동·뒷축 제동으로 느끼지 않게 재는 방법이 다음 단계

· 결과는 CARLA 시뮬레이션 · 이번 확장의 실시간 측정은 파이 격리 코어 대체 · 1차 카메라 경보 거리는 해상도가 한계

소스 코드 · 근거 문서 · 영상

github.com/gkgk0119gmail-arch/2026ESWContest_mobility_ARTS